Cours méthodes numériques (S.T. 2).

15 février 2012

Table des matières

1 Introduction. 3
- 1.1 Erreur absolue - Erreur relative. 3
- 1.2 Majorants des erreurs absolue et relative. 4
- 1.3 Chiffres exactes d'un nombre décimale. 4

2 Résolution numérique des équations algébriques et transcendantes. 5
- 2.1 Introduction. 5
- 2.2 Séparation des racines. 6
- 2.3 Méthode de Dichotomie (ou de Bissection) 6
- 2.4 Méthode de Newton – Raphson (ou de la tangente). 7
- 2.5 Méthode de la Sécante (ou de la fausse position). 8
  - 2.5.1 1er cas. 9
  - 2.5.2 2nd cas. 9
- 2.6 La méthode des approximations successives (ou du point fixe). 10

3 Approximation et interpolation polynômiale. 12
- 3.1 Introduction 12
- 3.2 Approximation polynômiale. 12
  - 3.2.1 Approximation aux sens des moindres carrées. 13
  - 3.2.2 Approximation aux sens des moindres carrées discrète. 14
- 3.3 Interpolation polynômiale. 14
  - 3.3.1 Introduction – Position du problème. 14
  - 3.3.2 Formule d'interpolation de Lagrange. 15
  - 3.3.3 Erreur de l'interpolation. 16

4 Intégration numérique. 17
- 4.1 Introduction 17

4.2 Approximation d'une intégrale définie. 17
4.3 Formules de quadrature de Newton–Côtes. 18
4.3.1 Méthode des trapèzes. 19
4.3.2 Méthode de Simpson (pour $n = 2p$). 20
4.4 Formule de Quadrature de Gauss. 21

5 Résolution des systèmes d'équations linéaires - Méthodes directes. 23
5.1 **Introduction.** 23
5.2 Méthode de l'élimination de Gauss. 25
5.3 Méthode de Jordan. 27
5.4 Méthode de décomposition LU. 28
5.5 Décomposition de Cholesky. 29

6 Résolution des systèmes d'équations linéaires - Méthodes itératives. 31
6.1 **Introduction** 31
6.2 Les méthodes itératives. 32
6.3 Convergence des méthodes itératives 33
6.4 Méthode de Jacobi 35
6.4.1 Estimation de l'erreur de la méthode de Jacobi. 35
6.5 Méthode de Gauss – Seidel 35
6.5.1 Estimation de l'erreur de la méthode de Gauss – Seidel. 36

7 Résolution numérique des équations différentielles ordinaires du premier ordre. 40
7.1 Introduction. 40
7.2 Méthode d'Euler. 41
7.3 Méthodes de Runge – Kutta. 43

Chapitre 1

Introduction.

L'objectif de l'analyse numérique est de concevoir et d'étudier des méthodes de résolution de certains problèmes de mathématiques issus de la modélisation des problèmes réels et dont on cherche à calculer la solution à l'aide d'un ordinateur, i. e. à trouver ou à donner des valeurs approchées de la solution exacte avec une certaine précision.

Le but de ce cours, est donc, de fournir les principes de base des méthodes numériques les plus utilisées.

1.1 Erreur absolue - Erreur relative.

Soient x une quantité exacte et x^* une valeur approché de x.
- Si $x^* > x$, x^* est dite valeur approchée de x par excés.
- Si $x^* < x$, x^* est dite valeur approchée de x par défaut.

Définition 1.1.1 *On appelle erreur absolue de x^* sur x la valeur notée par E et donnée par*

$$E = |x - x^*| .$$

Définition 1.1.2 *On appelle erreur relative de x^* sur x la valeur notée par E_r et donnée par*

$$E_r = \frac{|x - x^*|}{|x|} = \frac{E}{|x|}.$$

Si la valeur de x est connue on peut déterminer les valeurs de l'erreur absolue et l'erreur relative. Mais dans la majorité des cas, elle ne l'est pas Les erreurs absolue et relative deviennent donc inconnues, et pour les apprécier on introduit la notion du majorant de l'erreur.

1.2 Majorants des erreurs absolue et relative.

Définition 1.2.1 *On appelle majorant de l'erreur absolue d'une valeur approchée x^*, tout réel Δx vérifiant*

$$E = |x - x^*| \leq \Delta x \Leftrightarrow x^* - \Delta x \leq x \leq x^* + \Delta x.$$

Remarque 1.2.2 *· On prent on pratique le plus petit Δx possible.*
· Δx est appelé par abus de language erreur absolue de x^* sur x et on écrit dans ce cas : $x = x^* \pm \Delta x$ ou encore $x \approx x^* (\pm \Delta x)$

Définition 1.2.3 *On appelle majorant de l'erreur relative d'une valeur approchée x^* la quantité δx donnée par*

$$\delta x = \frac{\Delta x}{|x|},$$

appelé par un abus de language erreur relative.

1.3 Chiffres exactes d'un nombre décimale.

Définition 1.3.1 *Si l'erreur absolue vérifie*

$$\Delta x \leq 0.5 \times 10^{-m}$$

alors le chiffre correspondant à la $m^{ième}$puissance de 10 est dit significatif exacte, noté c.s.e., ainsi que tous ceux avant lui.

Exemple 1.3.2 *On obtient une approximation de π au moyen de la quantité $\frac{22}{7} = 3.142875...$, on en conclut que*

$$\Delta x = \left|\pi - \frac{22}{7}\right| = 0.00126... \approx 0.126 \times 10^{-2} \leq 0.5 \times 10^{-2},$$

donc le chiffre des centième est significatif et on a en tous 3 chiffres significatifs (3.14).

Chapitre 2

Résolution numérique des équations algébriques et transcendantes.

2.1 Introduction.

Si une équation algébrique ou transcendante est compliquée, il serai impossible qu'on puisse trouver ces racines avec précision. Par ailleurs, il est donc important d'étudier les méthodes numériques qui nous permettent d'obtenir une valeur approché de chacune de ces racines.

Dans ce chapitre, on se propose d'approcher numériquement les racines de l'équation

$$f(x) = 0 \tag{1.1}$$

où f est une fonction définie continue dans $[a, b]$ (domaine des racines).

Les méthodes numériques utilisées pour déterminer la valeur de la racine α de l'équation (1.1), nous donnent des suites itératives $\{x_n\}_{n\geq 0}$ (i. e. $x_{n+1} = F(x_n)$), qui convergent sous certaines conditions, vers α.

L'erreur absolue dans ce cas est notée par $e_n = |\alpha - x_n|$ et le test d'arrêt pour chaque méthode numérique est $|x_{n+1} - x_n| \leq \varepsilon =$ la précision voulue.

2.2 Séparation des racines.

Pour toutes les méthodes numériques qui suivent il faut d'abord faire la séparation des racines, c'est–à–dire trouver des intervalles de la forme $[a_i, b_i]$ dans lesquels la fonction f admet une racine est une seule. Pour cela, on fait appelle au théorème suivant.

Théorème 2.2.1 ***(Théorème des valeurs intermédiaires)***

Soit f une fonction définie continue dans l'intervalle $[a, b]$.

– Si $f(a).f(b) < 0$, alors il existe au moins une racine de f dans $]a, b[$, c'est–à–dire :

$$\exists\ c \in]a, b[\quad tq \quad f(c) = 0.$$

Et si en plus on a $f'(x) \neq 0$ pour tout $x \in]a, b[$, alors la racine est unique.

– Si $f(a).f(b) > 0$, alors f n'a pas da racine ou admet un nombre pair de racines dans $[a, b]$.

2.3 Méthode de Dichotomie (ou de Bissection)

Après avoir isolé une racine dans un intervalle $[a, b]$, on considère le point $x_0 = \dfrac{a+b}{2}$. On a deux cas :

$\underline{1^{er} \text{ cas :}}$ Si $f(x_0) = 0$, alors $\alpha = x_0$.

$\underline{2^{ème} \text{ cas :}}$ Si $f(x_0) \neq 0$, alors la racine α se trouve dans l'un des deux intervalles $[a, x_0], [x_0, b]$.

– Si $f(a).f(x_0) < 0$, alors $\alpha \in [a, x_0]$ et on pose dans ce cas $a_1 = a$ et $b_1 = x_0$.

– Si $f(b).f(x_0) < 0$, alors $\alpha \in [x_0, b]$ et on pose dans ce cas $a_1 = x_0$ et $b_1 = b$.

De cette manière on a obtenu un autre intervalle $[a_1, b_1]$ contenant α et dont la longueur est égale à la moitié de la longueur de $[a, b]$ qui est égale à $b - a$

Dans ce cas on considère le point $x_1 = \dfrac{a_1 + b_1}{2}$. On a deux cas :

$\underline{1^{er} \text{ cas :}}$ Si $f(x_1) = 0$, alors $\alpha = x_1$.

$\underline{2^{ème} \text{ cas :}}$ Si $f(x_1) \neq 0$, alors la racine α se trouve dans l'un des deux intervalles $[a_1, x_1], [x_1, b1]$.

– Si $f(a_1).f(x_1) < 0$, alors $\alpha \in [a_1, x_1]$, on pose $a_2 = a_1$ et $b_2 = x_1$.
– Si $f(b_1).f(x_1) < 0$, alors $\alpha \in [x_1, b_1]$, on pose $a_2 = x_1$ et $b_2 = b_1$.

De cette manière on a obtenu un autre intervalle $[a_2, b_2]$ contenant α et dont la longueur est égale à la moitié de la longueur de $[a_1, b_1]$ qui est égale à $\dfrac{b-a}{2}$.

En itérant le procédé, on obtient une suite de points

$$x_0 = \frac{a+b}{2}, x_1 = \frac{a_1+b_1}{2}, . \ . \ . \ , x_n = \frac{a_n+b_n}{2}, . \ . \ . \tag{1.2}$$

où x_0 est appelé **approximation initiale**, qui converge vers α.

Proposition 2.3.1 *L'estimation de l'erreur commise par la méthode de Dichotomie est donnée par*

$$e_n = |\alpha - x_n| \leq \frac{b-a}{2^{n+1}} \leq \varepsilon \tag{1.3}$$

et par la suite on a

$$n \geq \frac{\ln[(b-a)/\varepsilon]}{\ln 2} - 1. \tag{1.4}$$

2.4 Méthode de Newton – Raphson (ou de la tangente).

Soit $x_0 \in [a, b]$, tel que $f(x_0) \neq 0$. La tangente à la courbe $y = f(x)$ au point $M_0 = (x_0, f(x_0))$ coupe l'axe (ox) en un point d'abscisse x_1. La tangente à la courbe $y = f(x)$ au point $M_1 = (x_1, f(x_1))$ coupe l'axe (ox) en un point d'abscisse x_2, . . . etc.

En itérant le procédé on obtient une suite de points d'abscisses $\{x_n\}_{n\geq 0}$ définie par

$$\begin{cases} x_{n+1} = x_n - \dfrac{f(x_n)}{f'(x_n)} \quad , \ \text{pour tout } n \geq 1 \\ x_0 = \ \text{approximation initiale.} \end{cases} \tag{1.5}$$

applé **algorithme de Newton – Raphson**.

Remarque 2.4.1 *Une bonne approximation initiale x_0 est celle qui vérifie l'inégalité suivante*

$$f(x_0).f"(x_0) > 0. \tag{1.6}$$

Théorème 2.4.2 *Si $f \in C^2[a,b]$ et si f' et $f"$ gardent des des signes constants sur $[a,b]$, c–à–d : $f'(x) \neq 0$ et $f"(x) \neq 0$ sur $[a,b]$, alors la méthode de Newton – Raphson est dite convergente, i. e. $\lim_{n\to+\infty} x_n = \alpha$.*

Donc la racine exacte α de l'équation (1.1) peut être calculer à l'aide de la méthode de Newton – Raphson, avec la précision donnée et pour une approximation initiale vérifiant la condition (1.6).

Proposition 2.4.3 *L'estimation de l'erreur commise par la méthode de Newton – Raphson est donnée par*

$$e_n = |\alpha - x_n| \leq \frac{|f(x_n)|}{m_1}, \tag{1.7}$$

où $m_1 = \min_{[a,b]} |f'(x)|$.

Le test d'arrêt $|x_{n+1} - x_n| \leq \varepsilon \Leftrightarrow \left|\frac{f(x_n)}{f'(x)}\right| \leq \varepsilon$.

Remarque 2.4.4 ***1)*** *Si l'estimation* (1.7) *est mauvaise, alors on peut restreindre l'intervalle $[a,b]$.*

2) *Lorsque la méthode de Newton – Raphson converge, alors on a suivant le signe de $f"$:*

(a) $f" > 0$, alors si $f' > 0$ (respectivement $f' < 0$), la suite $\{x_n\}_{n\geq 0}$ est monotone et converge vers la racine α par excès, c–à–d $x_n > \alpha$ (respectivement par défaut).

(b) $f" < 0$, alors si $f' < 0$ (respectivement $f' > 0$), la suite $\{x_n\}_{n\geq 0}$ est monotone et converge vers la racine α par excès, c–à–d $x_n > \alpha$ (respectivement par défaut).

2.5 Méthode de la Sécante (ou de la fausse position).

Dans certains cas (si f provient d'un calcul expérimentale par exemple) on ne peut pas calculer f'. L'idée est alors de remplacer f' par le taux d'accroissement de f sur un petit intervalle.

Soit c un point de $[a,b]$ tel que $f'(c) \neq 0$. On choisie un point initial x_0 tel que $f(x_0).f(c) < 0$.

La sécante joingnant les points $M_c(c, f(c))$ et $M_0(x_0, f(x_0))$ coupe l'axe des abscisses en un point d'abscisse x_1. La sécante joingnant les points M_c et $M_1(x_1, f(x_1))$ coupe l'axe des abscisses en un point d'abscisse x_2.

En itérant le procédé on obtient une suite de points d'abscisses $\{x_n\}_{n\geq 0}$ définie par

$$x_{n+1} = x_n - f(x_n)\frac{x_n - c}{f(x_n) - f(c)} \quad , \quad \text{pour tout } n \geq 1. \tag{1.8}$$

Définition 2.5.1 *L'extrémité fixe c est celle dont le signe de f coïncide avec le signe de f", c–à–d* $f(x).f"(x) > 0$.

Soit pour fixer les idées, on pose $f"(x) > 0$ pour tout $x \in [a,b]$.

2.5.1 1er cas.

Si $f(a) > 0$, alors l'extrémité fixe $c = a$ et dans ce cas les approximations successives données par l'algorithme

$$\begin{cases} x_0 = b \\ x_{n+1} = x_n - f(x_n)\dfrac{x_n - a}{f(x_n) - f(a)} \quad , \quad \text{pour tout } n \geq 1, \end{cases} \tag{1.9}$$

forment une suite décroissante bornée

$$a < \alpha < \ . \ . \ . \ < x_{n+1} < x_n < \ . \ . \ . \ . < x_0 = b.$$

2.5.2 2nd cas.

Si $f(a) < 0$, alors l'extrémité fixe $c = b$ et dans ce cas les approximations successives données par l'algorithme

$$\begin{cases} x_0 = a \\ x_{n+1} = x_n - f(x_n)\dfrac{x_n - b}{f(x_n) - f(b)} \quad , \quad \text{pour tout } n \geq 1, \end{cases} \tag{1.10}$$

forment une suite croissante bornée

$$x_0 = a < x_1 < \ . \ . \ . \ < x_n < x_{n+1} < \ . \ . \ . \ . < \alpha < b.$$

Proposition 2.5.2 *L'estimation de l'erreur commise par la méthode de la sécante est donnée par*

$$e_n = |\alpha - x_n| \leq \frac{|f(x_n)|}{m_1}, \tag{1.11}$$

où $m_1 = \min_{[a,b]} |f'(x)|$.

2.6 La méthode des approximations successives (ou du point fixe).

L'équation (1.1) peut toujours s'écrire sous la forme

$$x = F(x). \tag{1.12}$$

Soit x_0 une approximation initiale quelconque, alors de (1.12) on obtient une suite $\{x_n\}_{n\geq 0}$ définie par

$$x_1 = F(x_0), x_2 = F(x_1), \cdots, x_{n+1} = F(x_n).$$

Si cette suite converge, alors on a $\alpha = \lim_{n\to+\infty} x_{n+1} = \lim_{n\to+\infty} F(x_n)$.

Ainsi, on peut dire que α est une racine de (1.12) qui se calcule en utilisant l'algorithme

$$\begin{cases} x_0 \quad \text{quelconque} \\ x_{n+1} = F(x_n), \end{cases} \tag{1.13}$$

avec la précision voulue.

Proposition 2.6.1 *On suppose que la fonction* $F\ I \to I$ *est une application contractente de rapport* k $(0 < k < 1)$, *i. e.*

$$\exists 0 < k < 1; |F(x_1) - F(x_2)| \leq k\,|x_1 - x_2|\,, \quad \textit{pour tout } x_1, x_2 \in I, \tag{1.14}$$

(si $k > 0$ *on dit que* F *est lipschitzienne), alors* F *admet un point fixe* α *(i. e.* $\alpha = F(\alpha)$*) et que la suite définie par l'algorithme* (1.13) *converge vers ce point fixe (racine exacte de l'équation* (1.1)*) et en plus on a l'estimation de l'erreur suivante*

$$|x_n - \alpha| \leq \frac{k^n}{1-k}\,|x_1 - x_0|\,. \tag{1.15}$$

Remarque 2.6.2 ***(1)*** *La condition* (1.14) *peut être remplacer par la condition suivante*

$$\exists 0 < k < 1; |F'(x)| \leq k, \quad \textit{pour tout } x \in I. \tag{1.16}$$

(2) *La formule* (1.15) *prévoit à l'avance le nombre d'itérations nécessaires à l'approximation de la racine* α.

En effet,

$$\begin{aligned} |x_n - \alpha| &\leq \frac{k^n}{1-k} |x_1 - x_0| \leq \varepsilon \\ &\Updownarrow \\ k^n &\leq \frac{1-k}{|x_1 - x_0|}\varepsilon \\ &\Updownarrow \\ n &\geq \frac{\ln\left[\dfrac{\varepsilon(1-k)}{|x_1 - x_0|}\right]}{\ln k}. \end{aligned} \tag{1.17}$$

(3) *Sous la condition* (1.14) *(ou* (1.16)*) la méthode des approximations successives converge quelque soit le choix de l'approximation initiale* x_0. *Il en résulte que cette méthode est* ***"auto correctrice".***

(4) *(a) Si* $0 \leq F'(x) \leq k < 1$, *alors la suite* $\{x_n\}_{n\geq 0}$ *converge vers* α *d'une façon monotone.*

(b) Si $1 < -k \leq F'(x) \leq 0$, *alors la suite* $\{x_n\}_{n\geq 0}$ *converge vers* α *alternativement par exés et par défaut.*

(5) *Si* $|F'(x)| > 1$, *la suite diverge.*

Chapitre 3

Approximation et interpolation polynômiale.

3.1 Introduction

Soit un ensemble fini de points x_i, $i = \overline{0,n}$, dans l'intervalle $[a,b]$. Nous supposerons qu'à chaque point x_i on peut associer un réel y_i (exp : résultat d'une expérience) telle que $y_i = f(x_i)$, mais que cette opération soit impossible pour tout $x \neq x_i$, $i = \overline{0,n}$. Nous voudrions déterminer donc une fonction facilement calculable, qui nous permet d'obtenir une estimation "raisonnable" de la réponse de ce phénomène pour tout $x \in [a,b]$.

Notons g cette fonction, nous avons donc 2 choix :

1- Nous pouvons chercher une fonction g d'un type préalablement choisi, telle que la **distance** entre les deux fonctions f et g soit minimale. C'est l'**approximation.**

2- Nous pouvons chercher une fonction g d'un type préalablement choisi, telle que $y_i = f(x_i) = g(x_i)$, pour tout $i = \overline{0,n}$. C'est l'**interpolation**.

3.2 Approximation polynômiale.

Définition 3.2.1 *L'approximation d'une fonction f par un polynômes est dite **approximation polynômiale**. Cette approximation est la plus utilisée,*

car les polynômes sont très faciles à dériver, à intégrer et constituent un espace vectoriel.

Théorème 3.2.2 *Si $f \in C[a,b]$, alors quelque soit $\varepsilon > 0$ il existe P_n un polynôme de degré n, telle que $\max_{a\leq x\leq b} |f(x) - P_n(x)| < \varepsilon$.*

3.2.1 Approximation aux sens des moindres carrées.

On définit sur l'espace vectoriel des fonctions continues sur $[a,b]$ le produit scalaire suivant :

$$\langle f, g\rangle = \int_a^b f(x)g(x)w(x)dx,$$

où w est une fonction positive continue par morceaux sur $[a,b]$. Dans ce cas, on peut définir une distance d dans cet espace de la manière suivante

$$d(f,g) = \sqrt{\langle f-g, f-g\rangle} \quad \text{pour tous } f, g \in C([a,b])$$

Théorème 3.2.3 *Une condition nécessaire et suffisante pour que la fonction g soit une meilleure approximation de la fonction f est*

$$\langle f-g, h\rangle = 0 \quad \textit{pour toute fonction continue } h. \tag{2.1}$$

La fonction g est appelée ***meilleure approximation de la fonction f au sens des moindres carrées.***

Si $g = \widetilde{P}_n \in \mathcal{P}_n$ qui est l'ensemble des polynômes de degré inferieur ou égale à n et comme les fonctions x^i, $i = \overline{0,n}$, forment une base de cette espace, alors on peut ecrire

$$\widetilde{P}_n(x) = \sum_{i=0}^{n} \widetilde{a}_i x^{n-i} \quad \text{et} \quad P_n(x) = \sum_{j=0}^{n} a_j x^{n-j}$$

pour tout polynôme P_n. La condition (2.1) devient dans ce cas

$$\left\langle f, x^{n-j}\right\rangle = \sum_{i=0}^{n} \widetilde{a}_i \left\langle x^{n-i}, x^{n-j}\right\rangle \qquad \text{pour tout } j = \overline{0,n}. \tag{2.2}$$

En remplaçant le produit scalaire par sa valeur on obtient

$$\int_a^b x^{n-j} f(x)w(x)dx = \sum_{i=0}^{n} \widetilde{a}_i \int_a^b x^{2n-i-j} w(x)dx \qquad \text{pour tout } j = \overline{0,n}. \tag{2.3}$$

3.2.2 Approximation aux sens des moindres carrées discrète.

La résolution de (2.3) nous permet de déterminer l'approximation au sens des moindres carrés pour une fonction f connue analytiquement. Cela n'est pas le cas si f n'est pas connues sur tout l'intervalle $[a, b]$ et ou on a la valeur de cette fonction dans un nombre fini de points $x_0, x_1, \quad . \quad . \quad ., x_m$.

Dans ce cas le produit scalaire sera défini de la manière suivante

$$\langle f, g\rangle = \sum_{k=0}^{m} f(x_k)g(x_k)w(x_k).$$

Donc (2.2) devient

$$\sum_{k=0}^{m} x_k^{n-j} f(x_k)w(x_k) = \sum_{i=0}^{n} \widetilde{a}_i \left[\sum_{k=0}^{m} x_k^{2n-i-j} w(x_k)\right] \tag{2.4}$$

pour tout $j = \overline{0, n}$.

3.3 Interpolation polynômiale.

3.3.1 Introduction – Position du problème.

Soit $y = f(x)$ une fonction dont on ne connait que les valeurs y_i qu'elle prend aux $n + 1$ points distincts x_i, $i = \overline{0, n}$. On a donc

$$y_i = f(x_i), \quad i = \overline{0, n}. \tag{2.5}$$

Les $(x_i, f(x_i))$ pour $i = 0, 1, 2, \cdots, n$, sont appelés **points d'interpolation** ou **points de collocation**.

Le problème d'interpolation consiste à trouver la fonction g (**appelée fonction d'interpollation**) qui appartient à une certaine classe de fonctions et qui prend aux points d'interpolation les mêmes valeurs que f, c–à–d

$$g(x_0) = y_0, g(x_1) = y_1, \quad \ldots, g(x_n) = y_n.$$

Dans notre cas on considère que g appartient à l'ensemble des polynômes de degré $\leq n$. On écrit donc

$$y_i = P_n(x_i), \quad i = \overline{0,n}. \tag{2.6}$$

C'est ce qu'on appele **interpoler** f par un polynôme P_n sur l'intervalle $[\min\{x_i\}, \max\{x_i\}]$ appelé **intervalle d'interpolation**.

P_n s'appele **polynômes d'interpolation de** f en $x_0, x_1, \ldots, x_n$.

Le problème posé sous sa forme générale peut avoir une infinité de solutions ou non avoir point. Toutefois, il existe une seule solution si on cherche un polynôme P_n.

3.3.2 Formule d'interpolation de Lagrange.

Essayons d'abord de résoudre le problème suivant :

$$\begin{cases} \text{Construire un polynôme} l_i(x) \text{ de degré } n \text{ et tel que} \\ l_i(x_j) = \delta_{ij} = \begin{cases} 0 & : i \neq j \\ 1 & : i = j. \end{cases} \end{cases} \tag{2.7}$$

où δ_{ij} est le symbole de **Kronicker.** Ces polynômes sont appelés les **polynômes de Lagrange**.

On en déduit d'après (2.7) que chaque polynôme $l_i(x)$ admet exactement n racines, qui sont $x_0, \cdots, x_{i-1}, x_{i+1}, \cdots, x_n$, et $l_i(x_i) = 1$, ce qui nous permet d'écrire que

$$l_i(x) = \frac{(x-x_0)\cdots(x-x_{i-1})(x-x_{i+1})\cdots(x-x_n)}{(x_i-x_0)\cdots(x_i-x_{i-1})(x_i-x_{i+1})\cdots(x_i-x_n)}$$

ou encore

$$l_i(x) = \prod_{\substack{j=0 \\ j\neq i}}^{n} \frac{x-x_j}{x_i-x_j}. \tag{2.8}$$

Passons maintenant à la résolution du problème générale, qui consiste à former le polynôme P_n vérifiant (2.6), noté par $\mathcal{L}_n(x)$ et qui est de la forme

$$\mathcal{L}_n(x) = \sum_{i=0}^{n} y_i l_i(x) \quad \text{pour tout } x \in [\min\{x_i\}, \max\{x_i\}]. \tag{2.9}$$

On a $\deg \mathcal{L}_n \leq n$ et $\mathcal{L}_n(x_j) = \sum_{i=0}^{n} y_i l_i(x_j) = y_j l_j(x_j) = y_j$.

Ainsi

$$\mathcal{L}_n(x) = \sum_{i=0}^{n} y_i \prod_{\substack{j=0 \\ j\neq i}}^{n} \frac{x - x_j}{x_i - x_j} \quad \text{pour tout } x \in [\min\{x_i\}, \max\{x_i\}]. \tag{2.10}$$

$\mathcal{L}_n$ s'appelle **le polynôme d'interpolation de Lagrange**.

Remarque 3.3.1 *Une fois la formule d'interpolation est obtenue, on l'utilise dans le calcul approché des valeurs de f pour $x \neq x_i$ pour $i = 0, 1, 2, \cdots, n$, i. e. $f(x) \approx \mathcal{L}_n(x)$ pour tout $x \in [\min\{x_i\}, \max\{x_i\}]$.*

3.3.3 Erreur de l'interpolation.

Dans la pratique l'interpolation polynômiale sert à remplacer une fonction inconnue f par une fonction simple qui est un polynôme. On dit que l'on approxime f par le polynôme d'interpolation. Il est fondamental alors, d'étudier l'erreur de l'interpolation.

Théorème 3.3.2 *Soit $[a, b]$ un intervalle contenant les points $x_0, x_1, \cdots, x_n$. On suppose que f est $(n+1)$ fois dérivable sur $]a, b[$. Alors pour tout $x \in [a, b]$ il existe $\xi = \xi_x \in]a, b[$ tel que*

$$E_n(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i). \tag{2.11}$$

Dans ce cas

$$|f(x) - P_n(x)| \leq \frac{M_{n+1}}{(n+1)!} \prod_{i=0}^{n} |x - x_i|, \tag{2.12}$$

où $M_{n+1} = \max_{[a,b]} \left|f^{(n+1)}(x)\right|$.

Chapitre 4

Intégration numérique.

4.1 Introduction

Soit f une fonction définie sur un intervalle $[a, b]$. Nous supposerons l'intégrale définie suivant

$$I = \int_a^b f(x)dx. \tag{3.1}$$

Il n'est pas toujours possible de connaître exactement la valeur de cette intégrale. En effet, on ne peut pas toujours trouver explicitement une primitive de f, même dans le cas ou f est donnée par une formule analytique explicite, encore moins si on ne connait que les valeurs de f en un nombre fini de points.

On se propose de chercher une approximation numérique de cette intégrale.

4.2 Approximation d'une intégrale définie.

Nous voulons donner une valeur approchée de l'intégrale définie (3.1). Supposons que f soit connue en $(n + 1)$ points $x_0, x_1, \cdots, x_n$. Nous allons écrire que

$$I = \sum_{i=0}^{n} \alpha_i f(x_i) + R \tag{3.2}$$

où les coefficients α_i seront choisis de telle sorte que le reste R soit nul lorsque f est d'un type déterminé.

Pour f quelconque on appellera la formule suivante

$$I \approx A_n = \sum_{i=0}^{n} \alpha_i f(x_i) \tag{3.3}$$

formule de quadrature et dans ce cas A_n sera une valeur approchée de I si R est suffisament petit. La question essentielle est de savoir si en augmentant le nombre de points x_i, on obtiendra une valeur de A_n de plus en plus proche de I, plus précisément, a – t – on

$$\lim_{n\to+\infty} A_n = \lim_{n\to+\infty} \sum_{i=0}^{n} \alpha_i f(x_i) = I. \tag{3.4}$$

La réponse est positive que sous certaines conditions sur les α_i et les x_i.

Théorème 4.2.1 *Lorsque f est continue sur $[a, b]$, nous avons* (3.4) *si les conditions suivantes sont vérifiées.*

1) (3.4) *est vrai pour tout polynôme P.*

2) $\sum_{i=0}^{n} |\alpha_i|$ *est bornée pour tout n.*

4.3 Formules de quadrature de Newton–Côtes.

Dans ce paragraphe, nous allons supposer que les x_i sont équidistants (i.e. $x_{i+1} - x_i = h = const$) et que l'on a

$$x_0 = a \text{ et } x_i = x_{i-1} + h = a + ih \text{ pour } i = 1, ..., n.$$

Dans ce cas $h = \dfrac{b-a}{n}$.

En interpolant f par des polynômes de degré m par morceaux, on obtient les formules de quadrature de Newton–Côtes suivantes

$$\int_a^b f(x)dx \approx (b-a) \sum_{i=0}^{m} H_i f(x_i) \tag{3.5}$$

où

$$H_i = \frac{(-1)^{m-i}}{i!(m-i)!m} \int_0^m \frac{q^{[m+1]}}{q-i} dq$$

et $q^{[m+1]} = q(q-1)(q-2)\cdots(q-m)$, appelé **puissance généralisée**.

4.3.1 Méthode des trapèzes.

En interpolant f par un polynôme de degré 1 sur le sous intervalle $[x_0, x_1]$ (i.e. on remplace m par 1 dans (3.5)) on peut écrire que

$$\int_{x_0}^{x_1} f(x)dx \approx h(H_0 y_0 + H_1 y_1)$$

avec $H_0 = H_1 = 1/2$. Donc

$$\int_{x_0}^{x_1} f(x)dx \approx \frac{h}{2}(y_0 + y_1).$$

De la même manière nous pouvons écrire

$$\int_{x_i}^{x_{i+1}} f(x)dx \approx \frac{h}{2}(y_i + y_{i+1}), \text{ pout tout } i = \overline{0, n-1}.$$

Comme $I = \int_a^b f(x)dx = \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} f(x)dx$, alors la formule des trapèzes est

$$I = \frac{b-a}{n}\left[\frac{y_0}{2} + y_1 + y_2 + \cdots \frac{y_0}{2}\right] + R_T \tag{3.6}$$

où R_T est l'erreur commise par cette méthode vérifiant

$$\begin{aligned} |R_T| &= \left| -\frac{h^2}{12}(b-a)f''(\xi) \right|, \xi \in]a, b[\\ &\leq \frac{nh^3}{12} \max_{a \leq x \leq b} |f''(x)| \end{aligned} \tag{3.7}$$

et dans ce cas on dit que l'erreur de la formule des trapèzes est d'ordre 2 en h.

4.3.2 Méthode de Simpson (pour $n = 2p$).

En interpolant f par un polynôme de degré 2 sur le sous intervalle $[x_0, x_2]$ (i. e. on remplace m par 2 dans (3.5)) on peut écrire que

$$\int_{x_0}^{x_2} f(x)dx \approx 2h(H_0 y_0 + H_1 y_1 + H_2 y_2)$$

avec $H_0 = H_2 = 1/6$ et $H_1 = 2/3$. D'où

$$\int_{x_0}^{x_2} f(x)dx \approx \frac{h}{3}(y_0 + 4y_1 + y_2).$$

De la même manière nous pouvons écrire

$$\int_{x_{2i}}^{x_{2i+2}} f(x)dx \approx \frac{h}{3}(y_{2i} + 4y_{2i+1} + y_{2i+2})$$

En sommant nous obtiendrons pour $n = 2p$

$$\begin{aligned} I &= \int_a^b f(x)dx = \sum_{i=0}^{p-1} \int_{x_{2i}}^{x_{2(i+1)}} f(x)dx \\ &\approx \frac{h}{3}\left[y_0 + 4\left(y_1 + y_3 + \cdots y_{n-1}\right) + 2\left(y_2 + y_4 + \cdots y_{n-2}\right) + y_n\right]. \end{aligned}$$

En posant $\sigma_1 = y_1 + y_3 + \cdots y_{n-1}$ et $\sigma_2 = y_2 + y_4 + \cdots y_{n-2}$ on obtient la formule de Simpson suivante

$$I = \frac{h}{3}\left[y_0 + 4\sigma_1 + 2\sigma_2 + y_n\right] + R_S \tag{3.8}$$

où R_S est l'erreur commise par cette méthode avec

$$|R_S| \leq \frac{(b-a)^5}{180n^4} \max_{a\leq x\leq b} \left|f^{(4)}(x)\right| = \frac{(b-a)h^4}{180} \max_{a\leq x\leq b} \left|f^{(4)}(x)\right| \tag{3.9}$$

et dans ce cas on dit que l'erreur de la formule de Simpson est d'ordre 4 en h.

4.4 Formule de Quadrature de Gauss.

Définition 4.4.1 *On appele polynômes de Legendre les expressions de la forme*

$$P_n(x) = \frac{1}{2^n n!}\frac{d^n}{dx^n}\left[(x^2-1)^n\right], \qquad n \geq 0 \tag{3.10}$$

Voici les propriétés fondamentales de ces polynômes

1) $P_n(1) = 1,\ P_n(-1) = (-1)^n$ pour tout $n \geq 0$.
2) $\deg P_n = n$ pour tout $n \geq 0$.
3) $\int_{-1}^{1} x^k P_n(x)dx = 0,\ k < n \Leftrightarrow \int_{-1}^{1} P_m(x)P_n(x)dx = 0$ pour tout $m \neq n$. On dit dans ce cas que ces polynômes sont orthogonaux par rapport à la fonction poids $w(x) = 1$(i.e. $\int_a^b P_m(x)P_n(x)w(x)dx = k_n\delta_{mn}$).
4) Le polynôme de Legendre $P_n(x)$ possède n racines distintes et réelles comprise entre $[-1, 1]$.

On cite ci–dessous les 5 premiers polynômes

$$\begin{aligned} P_0(x) &= 1, \\ P_1(x) &= x, \\ P_2(x) &= 1/2(3x^2-1), \\ P_3(x) &= 1/2(5x^3-3x) \text{ et} \\ P_4(x) &= 1/8(35x^4-30x^2+3). \end{aligned}$$

Considérons d'abord que la fonction $y = f(t)$ est définie sur $[-1, 1]$. Le problème est de trouver une formule de quadrature (i.e trouver les A_i et les t_i) de la forme

$$\int_{-1}^{1} f(t)dt \approx \sum_{i=1}^{n} A_i f(t_i) \tag{3.11}$$

qui soit exacte pour tout polynômes dont le degré est le plus grand possible.

En on déduit alors que pour résoudre ce problème il suffit de déterminer les t_i et les A_i à partir du système d'équations suivant

$$\sum_{i=1}^{n} A_i t_i^k = \int_{-1}^{1} t^k dt = \begin{cases} \dfrac{2}{k+1} & ; k \text{ pair} \\ 0 & ; k \text{ impair} \end{cases}, \ , \qquad k = 0, \cdots, n-1,$$

où les t_i sont les zéros du polynôme de Legendre de degré n. La formule obtenue ainsi, est appelée **formule de quadrature de Gauss à n coordonnées.**

Pour f définie sur un intervalle quelconque $[a, b]$, on considère le changement de variable suivant

$$x = \frac{b-a}{2}t + \frac{b+a}{2}, \text{ pour tout } t \in [-1, 1],$$

qui transforme l'intervalle $[-1, 1]$ à $[a, b]$. Donc on peut écrire que la formule de quadrature de Gauss à n ordonnées sur $[a, b]$ est donnée par

$$\int_a^b f(x)dx \approx \frac{b-a}{2} \sum_{i=1}^{n} A_i f(x_i) \tag{3.12}$$

où $x_i = \frac{b-a}{2}t_i + \frac{b+a}{2}$, pour tout $i = \overline{1, n}$

Chapitre 5

Résolution des systèmes d'équations linéaires - Méthodes directes.

5.1 Introduction.

Dans ce chapitre, on se propose d'étudier quelques méthodes permettant de résoudre les systèmes de n équations à n inconnues de la forme

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n. \end{cases} \tag{4.1}$$

Soient $A = (a_{ij}) \in M_n(\mathbb{R})$ (l'ensemble des matrices carrées d'ordre n à éléments dans $\mathbb{R}$), B le vecteur de composantes $(b_1, b_2, \cdots, b_n)$ et X le vecteur inconnu de composantes $(x_1, x_2, \cdots, x_n)$. Résoudre le système (4.1) revient alors a résoudre l'équation matricielle

$$AX = B. \tag{4.2}$$

Pour résoudre ce genre de système, la première idée qui nous vient à

l'esprit est l'utilisation des formules de Cramer données par

$$x_i = \frac{1}{\det A} \begin{vmatrix} a_{11} & \dots & a_{1,i-1} & b_1 & a_{1,i+1} & \dots & a_{1n} \\ a_{21} & \dots & a_{2,i-1} & b_2 & a_{2,i+1} & \dots & a_{2n} \\ \vdots & & \vdots & \vdots & \vdots & & \vdots \\ \vdots & & \vdots & \vdots & \vdots & & \vdots \\ a_{n1} & \dots & a_{n,i-1} & b_n & a_{n,i+1} & \dots & a_{nn} \end{vmatrix}. \tag{4.3}$$

Il en résulte que pour la résolution du système (4.1) par ces formules nécessite au total $T_{Cramer} = (n+1)^2 n! - 1$ opérations élémentaires, par exemple pour $n = 5$, on a $T_{Cramer} = 4319$ et pour $n = 10$, on a $T_{Cramer} \backsim 4.10^8$.

Dans ce cas, elle devient trés lente en temps d'execution (de calculs) dés que n dépasse 4.

Une autre méthode consiste à calculer la matrice inverse de A, c'est–à–dire :

$$A^{-1}A\ X\ = A^{-1}B \text{ d'où } X = A^{-1}B. \tag{4.4}$$

Pour cela on utilise au total $T_{Inverse} = n!(n^2 + n + 1) + 3n^2 - n$ opérations élémentaires, par exemple pour $n = 5$, on a $T_{Inverse} = 3790$ et pour $n = 10$, on a $T_{Inverse} \backsim 4.10^8$.

Cette méthode n'est donc pas plus avantageuse que la première.

Dans ce chapitre, nous fairions appelle à des méthodes numériques ayant des temps de calcul acceptables et dont le nombre d'opérations est d'ordre n^3. Ces méthodes peuvent être :

- Directes qui sont utilisées seulement si le nombre d'équations n'est pas trop élevé ($n \leq 100$).

- Iteratives qui sont utilisées dès que la taille du système est grande ($n \geq 100$), ce qui ne les empèches pas d'être également utiles pour des petits systèmes.

Définition 5.1.1 *On appelle méthode directe de résolution du système* (1)*, une méthode qui aboutit à la* ***solution exacte*** *du problème au bout d'un* ***nombre fini*** *d'opérations arithmétiques, en fonction de la dimension du système.*

Si la matrice A du système (4.1) est triangulaire, sa résolution est immédiate. En effet, si A est triangulaire supérieure, le système (4.1) peut s'écrire

sous la forme suivante

$$\left\{\begin{array}{lllllll} a_{11}x_1+ & a_{12}x_2+ & \ldots+ & a_{in}x_i+ & \ldots & a_{1n}x_n & = & b_1 \\ & a_{22}x_2+ & \ldots+ & a_{i2}x_i+ & \ldots & a_{2n}x_n & = & b_2 \\ & & \ddots & \vdots & & \vdots & \vdots & \vdots \\ & & & a_{ii}x_i+ & \ldots & a_{in}x_n & = & b_i \\ & & & & \ddots & \vdots & \vdots & \vdots \\ & & & & & a_{nn}x_n & = & b_n \end{array}\right. \tag{4.5}$$

Si $\det A \neq 0$, c'est–à–dire $a_{ii} \neq 0$ pour tout $i = 1, \ldots, n$, l'algorithme de résolution s'écrira

$$\left\{\begin{array}{l} x_n = b_n/a_{nn} \\ x_i = \left(b_i - \displaystyle\sum_{j=i+1}^{n} a_{ij}x_j\right)\frac{1}{a_{ii}}. \end{array}\right. \tag{4.6}$$

Et si A est triangulaire inférieure, le système (4.1) peut s'écrire sous la forme suivante

$$\left\{\begin{array}{lllllll} a_{11}x_1 & & & & & = & b_1 \\ a_{21}x_1+ & a_{22}x_2 & & & & = & b_2 \\ \vdots & & \ddots & & & \vdots & \vdots \\ a_{i1}x_1+ & a_{i2}x_2+ & \ldots+ & a_{ii}x_i & & = & b_i \\ \vdots & \vdots & & \vdots & \ddots & \vdots & \vdots \\ a_{n1}x_1+ & a_{n2}x_2+ & \ldots+ & a_{ni}xi & +\ldots \quad +a_{nn}x_n & = & b_n \end{array}\right. \tag{4.7}$$

dans ce cas l'algorithme de résolution devient si $\det A \neq 0$

$$\left\{\begin{array}{l} x_1 = b_1/a_{11} \\ x_i = \left(b_i - \displaystyle\sum_{j=1}^{i-1} a_{ij}x_j\right)\frac{1}{a_{ii}}. \end{array}\right. \tag{4.8}$$

pour cela on utilise au total n^2 opérations élémentaires.

5.2 Méthode de l'élimination de Gauss.

Théorème 5.2.1 ***Théorème de Gauss.*** *Etant donnée une matrice carrée A, il existe une matrice inversible M , telle que la matrice $A' = M.A$ soit triangulaire supérieure.*

La méthode de Gauss transforme le système (4.1) en un système triangulaire équivalent au système (4.5), à l'aide d'un algorithme d'élimination, en utilisant seulement les trois opérations élémentaires suivantes :

1. Opération ($\boldsymbol{l}_i \leftarrow \boldsymbol{\lambda l}_i$) : remplacer la ligne i par un multiple d'elle même.

2. Opération ($\boldsymbol{l}_i \leftarrow \boldsymbol{l}_j$) : intervertir la ligne i et la ligne j.

3. Opération ($\boldsymbol{l}_i \leftarrow \boldsymbol{l}_i + \boldsymbol{\lambda l}_j$) : remplacer la ligne i par la ligne i plus un multiple de la ligne j.

Ces 3 opérations élémentaires sont permise, car elle nous permettent d'obternir un système triangulaire équivalent au système (4.1) (i. e. ayant la même solution).

Exemple 5.2.2 *Voir au cours.*

Remarque 5.2.3 *Il peut arriver que lors de l'élimination, dans l'ordre naturelle des lignes, un pivot soit nul, en outre pour des raisons de stabilité, on a intérêt, à chaque étape d'élimination à choisir le pivot telle que* $\left|a_{kk}^{(k)}\right|$ *soit le plus grand possible. C'est la méthode de Gauss à un pivot partiel.*

La méthode de l'élimination de Gauss consiste à éliminer tous les termes sous la diagonale de la matrice A en utilisant au total $T_{Gauss} = (4n^3 + 9n^2 - 7n)/6$ opérations élémentaires, par exemple pour $n = 5$, $T_{Gauss} = 115$ et pour $n = 10$, $T_{Gauss} = 805$. Ce qui montre l'énorme amélioration que cette méthode apporte par rapport aux deux autres méthodes.

Remarque 5.2.4 *La méthode de Gauss permet de calculer le déterminant de la matrice A, et ceci en utilisant le faite que*

$$\det A = (-1)^p \prod_{k=1}^{n} a_{kk}^{(k)}, \tag{4.9}$$

où p est le nombre de permutations.

Algorithme de Gauss

1- Triangularisation

Pour $k \leq i$ et pour $i = 1,\ldots,n$, on pose $a_{kj}^{(i+1)} = a_{kj}^{(i)}$ et $b_k^{(i+1)} = b_k^{(i)}$.

Pour $k > i$,si $a_{ii}^{(i)} \neq 0$,on pose

$$\begin{cases} a_{kj}^{(i+1)} = a_{kj}^{(i)} - \dfrac{a_{ki}^{(i)}}{a_{ii}^{(i)}} a_{ij}^{(i)}, \text{pour } k = j, ..., n \\ b_k^{(i+1)} = b_k^{(i)} - \dfrac{a_{ki}^{(i)}}{a_{ii}^{(i)}} b_i^{(i)} \end{cases} \tag{4.10}$$

2- Résolution

$$x_i = \left(b_i^{(n)} - \sum_{j=i+1}^{n} a_{ij}^{(n)} x_j \right) \Big/ a_{ii}^{(n)}. \tag{4.11}$$

5.3 Méthode de Jordan.

Théorème 5.3.1 ***Théorème de Jordan.*** *Soit $A = (a_{ij}) \in M_n(\mathbb{R})$. Alors il dxiste des matrice S telle que la matrice $D = S.A$ soit diagonale d'ordre n.*

La méthode de Jordan consiste donc, à transformer le système (1) en un système à matrice diagonale $DX = B' \Leftrightarrow X = D^{-1}B'$.

Algorithme de Jordan

Transformation $(A/B) \backsim \left(I/D^{-1}B'\right)$

$$\begin{cases} a_{kk}^{(k)} = a_{kj}^{(k-1)} / a_{kk}^{(k-1)}, & j = k+1, \ldots, n+1 \\ a_{ij}^{(k)} = a_{ij}^{(k-1)} - a_{ik}^{(k-1)}.a_{kj}^{(k)}, & j = k+1, \ldots, n+1 \end{cases} \tag{4.12}$$

pour i et $k = 1, \ldots, n$ et $i \neq k$.

Remarque 5.3.2 *On peut calculer la matrice inverse de A par cette méthode et ceci en écrivant $(A/I) \backsim \left(I/A^{-1}\right)$.*

5.4 Méthode de décomposition LU.

Ces méthodes consiste à décomposer la matrice A en écivant $A = L.U$, ou L est une matrice triangulaire inférieure et U est une matrice triangulaire supérieure. Le sytème (4.1) s'écrit alors

$$AX = B \Leftrightarrow L\,(UX) = B, \tag{4.13}$$

qui peut se décomposer en 2 systèmes trangulaires

$$\begin{cases} LY = B \\ UX = Y. \end{cases} \tag{4.14}$$

Théorème 5.4.1 *Une condition nécessaire et suffisante pour qu'une matrice inversible A puisse se décomposer sous la forme $A = L.U$ est que*

$$\det A_k \neq 0, \text{ pour tout } k = 1, \ldots, n-1,$$

avec $A_k = (a_{ij})_{i,j=1}^{k}$ (i.e. les mineurs sont tous non nuls)

Remarque 5.4.2 *Il faut souligner que la décomposition LU n'est pas unique, donc il faut faire au préalable des choix arbitraires. Les deux choix les plus populaires consistent à imposer que la matrice U ait des 1 sur la diagonale (décomposition de Crout) ou bien que la matrice L ait des 1 sur la diagonale (décomposition de Doolittle).*

Algorithme de Doolittle.

$$\begin{cases} l_{ii} = 1 & i = 1, \ldots, n \\ u_{rj} = a_{rj} - \displaystyle\sum_{k=1}^{r-1} l_{rk} u_{kj} & j = r, \ldots, n \qquad r = 1, \ldots, n \\ l_{ir} = \left(a_{ir} - \displaystyle\sum_{k=1}^{r-1} l_{ik} u_{kr} \right) \Big/ u_{rr} & i = r+1, \ldots, n \quad r = 1, \ldots, n \end{cases} \tag{4.15}$$

Algorithme de Crout.

$$\begin{cases} u_{ii} = 1 & i = 1, \ldots, n \\ l_{ir} = a_{ir} - \sum_{k=1}^{r-1} l_{ik} u_{kr} & i = r, \ldots, n \qquad r = 1, \ldots, n \\ u_{rj} = \left(a_{rj} - \sum_{k=1}^{r-1} l_{rk} u_{kj} \right) \Big/ l_{rr} & j = r+1, \ldots, n \quad r = 1, \ldots, n \end{cases} \tag{4.16}$$

Remarque 5.4.3 *Le déterminant de la matrice A peut–être calculer aisément, en effet,*

$$\det A = \det L . \det U$$

pour la méthode de Doolittle comme $\det A = 1$*, on a*

$$\det A = \det U = \prod_{k=1}^{n} u_{kk}$$

et pour la méthode de Crout comme $\det U = 1$*, on a*

$$\det A = \det L = \prod_{k=1}^{n} l_{kk}.$$

5.5 Décomposition de Cholesky.

Théorème 5.5.1 ***Théorème de Cholesky.*** *Si A est une matrice symétrique définie positive, alors elle peut se décomposer sous la forme*

$$A = L.L^T$$

ou L est une matrice réelle triangulaire inférieure.

Définition 5.5.2 *Une matrice A est dite définie positive si*

$$\langle Ax, x \rangle > 0 \text{ , pour tout vecteur } x \in \mathbb{R}^n$$

ou $\langle ., . \rangle$ désigne le produit scalaire dans $\mathbb{R}^n$, c'est –à–dire que pour tout x et y dans $\mathbb{R}^n$ on a $\langle x, y \rangle = \sum_{i=1}^{n} x_i y_i$.

Supposant que A est une matrice symétrique définie positive, alors on a

$$AX = B \Leftrightarrow L.\left(L^T X\right) = B \Leftrightarrow \begin{cases} LY = B \\ L^T X = Y. \end{cases} \tag{4.17}$$

Le problème consiste donc à construire explicitement la matrice L.

L'algorithme de factorisation

$$\begin{cases} l_{rr} = \sqrt{a_{rr} - \sum_{k=1}^{r-1} l_{rk}^2}, & r = 1, \ldots, n \\ l_{jr} = \left(a_{rj} - \sum_{k=1}^{r-1} l_{rk} l_{kj} \right) / l_{rr}, & j = r+1, \ldots, n \quad \text{et } r = 1, \ldots, n \end{cases} \tag{4.18}$$

La méthode de Cholesky utilise

$$\begin{cases} (n^3 + 6n^2 - 7n)/6 & \text{additions et soustractions} \\ n^2 + 3n/2 & \text{divisions} \\ n & \text{racines carrées,} \end{cases}$$

soit au total $T_{Cholesky} = (2n^3 + 15n^2 + n)/6$ opérations élémentaires, par exemple pour $n = 5$, $T_{Cholesky} = 105$ et pour $n = 10$, $T_{Cholesky} = 585$.

Remarque 5.5.3 *Si la matrice A n'est pas définie positive, alors on a au moins un des $l_{kk}^2 < 0$.*

Chapitre 6

Résolution des systèmes d'équations linéaires - Méthodes itératives.

6.1 Introduction

Dans ce chapitre on considère le système (4.2) qui est un système d'équations linéaires d'ordre N . Nous avons vu que pour résoudre numériquement ce système, il suffit de procéder à une élimination de Gauss avec ou sans changement de pivot, ou eventuellement, de procéder à une décomposition LU de A (ou LL^T si A est symétrique définie positive), puis résoudre deux systèmes triangulaires.

Dans le cas ou la matrice A est pleine, le nombre d'opérations nécessaires à la mise en oeuvre de ces algorithmes est de l'odre N^3, ce qui peut être énorme lorsque N est grand. Dans ces conditions il devient beaucoup plus commode de rechercher les racines de ce système en utilisant les méthodes numériques approchées dites **méthodes itératives**.

Le principe de ces méthodes est de construire une suite de vecteurs $x^{(0)}, x^{(1)}, ..., x^{(n)}, ...$, définie par

$$\begin{cases} x^{(0)} \text{ vecteur initial} \\ x^{(n+1)} = \boldsymbol{B}x^{(n)} + C \end{cases} \tag{5.1}$$

ou B esty une matrice d'ordre $N \times N$ dite matrice d'itération et C est un vecteur de $\mathbb{R}^N$ telle que

$$\lim_{n\to\infty} \left\|x - x^{(n)}\right\| = 0. \tag{5.2}$$

On dit dans ce cas que la suite$\left\{x^{(n)}\right\}$ converge vers la la solution exacte de systèe (4.1) et on peut ecrire

$$x = \boldsymbol{B}x + C.$$

Donc cette équation est du type point fixe.

Remarquons au passage que les indices i de $x^{(i)}$ sont placés en haut du symbole x pour ne pas le confondre la i-ième composante x_i.

Remarque 6.1.1 *L'utilisation pratique de ces méthodes nécéssite non seulement la convergence,mais aussi que cette convergence ne soit pas trop lente.*

6.2 Les méthodes itératives.

Les méthodes de décomposition sont un premier exemple des méthodes itératives. Supposons que l'on est décomposé A en une difference de deux matrices carrée d'ordre N, K et M, c'est à dire

$$A = K - M \tag{5.3}$$

Si K est une matrice régulière, nous pouvons transformer (4.1) en utilisant (5.3) de la façon suivante :

$$(K - M)\, x = b \Longleftrightarrow Kx = Mx + b$$

soit encore

$$x = K^{-1}Mx + K^{-1}b \tag{5.4}$$

et dans ce cas on peut prendre $\boldsymbol{B} = K^{-1}M$ et $C = K^{-1}b$. Donc (5.1) devient

$$\begin{cases} x^{(0)} \text{ valeur initiale donnée} \\ x^{(n+1)} = K^{-1}Mx^{(n)} + K^{-1}b \end{cases} \tag{5.5}$$

Nous ferons en sorte que la décomposition (5.3) donne lieu à une matrice K telle que le système (5.5) soit facile à résoudre, ce qui est par exemple le

cas si K est diagonale ou triangulaire. Pour cela écrivons la matrice A sous la forme

$$A = D - E - F, \tag{5.6}$$

où D est la matrice diagonale formée des éléments $(a_{ii})_{1\leq i\leq N}$, $-E$ est la matrice formée des éléments sous-diagonaux de A

$$-E = \begin{pmatrix} 0 & 0 & \dots & \dots & 0 \\ a_{21} & 0 & & \dots & 0 \\ a_{31} & a_{32} & 0 & \ddots & \vdots \\ \vdots & & \ddots & \ddots & 0 \\ a_{N1} & a_{N2} & \dots & a_{N(N-1)} & 0 \end{pmatrix}$$

et $-F$ est la matrice formée des éléments surdiagonaux de A,

$$-F = \begin{pmatrix} 0 & a_{12} & \dots & \dots & a_{1N} \\ 0 & 0 & a_{23} & & a_{2N} \\ \vdots & & \ddots & \ddots & \vdots \\ \vdots & \vdots & \ddots & 0 & a_{(N-1)N} \\ 0 & 0 & & \dots & 0 \end{pmatrix}$$

Schèmatiquement nous avons

$$A = \begin{pmatrix} . & . & . & . & . \\ . & . & . & -F & . \\ . & . & D & . & . \\ . & -E & . & . & . \\ . & . & . & . & . \end{pmatrix}$$

Si nous supposons que $a_{ii} \neq 0$ pour tout $i = 1, 2, ..., N$, alors D et $(D-E)$ sont régulières et nous pouvons choisir par exemple $K = D$ (**méthode de Jacobi**) ou $K = D - E$ (**méthode de Gauss –Seidel**).

6.3 Convergence des méthodes itératives

Définition 6.3.1 *On dit que la méthode (le processus) itérative* (5.1) *converge pour tous vecteur C si* (5.2) *est vérifiée pour toute valeur initiale $x^{(0)}$.*

Définition 6.3.2 *Soit A est une matrice carrée d'ordre m, on appele rayon spectral de A la quantité suivante :*

$$\varrho(A) = \max_{1 \leq j \leq m} |\lambda_j|$$

où $\lambda_1, \lambda_2, ...\lambda_m$ sont les valeurs propres de la matrice A

Nous donnons sans démonstration les résultats suivants :

Théorème 6.3.3 *La méthode itérative* (5.2) *converge si $\varrho(\boldsymbol{B}) < 1$.*

Théorème 6.3.4 *Si A est une matrice symétrique alors*

$$\varrho(A) = \|A\|_e = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}$$

où $\|x\|_e = \sqrt{\sum_{i=1}^{m} x_i^2}$ est la norme matricielle induite de la norme vectorielle euclidienne.

Définition 6.3.5 *Soit $A = (a_{ij})_{i,j=1}^{n}$ est une matrice carrée d'ordre m, alors on peut définir les normes matricielles suivantes*

$$\|A\|_c = \max_i \sum_{j=1}^{m} |a_{ij}|, \tag{5.7}$$

$$\|A\|_l = \max_j \sum_{i=1}^{m} |a_{ij}| \tag{5.8}$$

et

$$\|A\|_r = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{m} a_{ij}^2} \tag{5.9}$$

Théorème 6.3.6 *Si pour une morme matricielle donnée on a $\|\boldsymbol{B}\| < 1$ alors la métgode itérative* (5.1) *est convergente.*

Définition 6.3.7 *Soit $A = (a_{ij})_{i,j=1}^{n}$ est une matrice carée d'ordre m. On dit que A est une matrice* ***à diagonale strictement dominante*** *si*

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \qquad ou \qquad |a_{jj}| > \sum_{i \neq j} |a_{ij}|.$$

Théorème 6.3.8 *Si la matrice A du système* (4.2) *est à diagonale strictement dominante alors toute méthode itérative de résolution est convergente.*

6.4 Méthode de Jacobi

Revenons aux relation (5.3) et (5.6). Si on pose $K = D$ et $M = E + F$, on peut ecrire le système (4.1) comme suit :

$$a_{ii}x_i = b_i - \sum_{j<i} a_{ij}x_j - \sum_{j>i} a_{ij}x_j \ , \quad \text{pour tout } i = 1, 2, \cdots N.$$

Donc on peut écrire la relation (5.5) composante par composante de la manière suivante :

$$x_i^{(n+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j\neq i}} a_{ij}x_j^{(n)} \right) \quad , \quad 1 \leq i \leq N. \tag{5.10}$$

Cette relation permet de calculer explicitement les composantes $x_i^{(n+1)}, i = 1, 2, ..., N$, du vecteur $x^{(n+1)}$ à partir des composantes $x_i^{(n)}, i = 1, 2, ...N$, du vecteur $x^{(n)}$. La matrice J définie par

$$J = K^{-1}M = D^{-1}(E + F)$$

est appelée **matrice de Jacobi.**

6.4.1 Estimation de l'erreur de la méthode de Jacobi.

L'estimation de l'erreur dû à la méthode de Jacobi vérifie

$$\left\|x - x^{(n)}\right\| \leq \frac{\|J\|^n}{1 - \|J\|} \left\|x^{(1)} - x^{(0)}\right\| \tag{5.11}$$

6.5 Méthode de Gauss − Seidel

En posant $K = D - E$ et $M = F$, le système (5.1) prend la forme

$$(D - E)\, x^{(n+1)} = Fx^{(n)} + b$$

qui est un système triangulaire inférieur pour l'inconnue $x^{(n+1)}$. Donc on peut écrire

$$a_{ii}x_i^{(n+1)} + \sum_{j=1}^{i-1} a_{ij}x_j^{(n+1)} = b_i - \sum_{j=i+1}^{N} a_{ij}x_j^{(n)} \ , \quad \text{pour tout } i = 1, 2, \cdots N.$$

Par conséquent la relation (5.5) sera écrite composante par composante de la manière suivante :

$$x_i^{(n+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(n+1)} - \sum_{j>i} a_{ij} x_j^{(n)} \right) \quad , \quad 1 \leq i \leq N. \tag{5.12}$$

Si les composantes $(x_i^{(n)})_{1 \leq i \leq N}$ du vecteur $x^{(n)}$ sont connues, les relations (5.12) permettent de calculer successivement $x_1^{(n+1)}, x_2^{(n+1)}, ...$, de la manière suivante :

$$\begin{cases} x_1^{(n+1)} = \frac{1}{a_{11}} \left(- \sum_{j=2}^{n} a_{1j} x_j^{(n)} + b_1 \right) \\ x_2^{(n+1)} = \frac{1}{a_{22}} \left(-a_{21} x_1^{(n+1)} - \sum_{j=3}^{n} a_{2j} x_j^{(n)} + b_2 \right) \\ x_3^{(n+1)} = \frac{1}{a_{33}} \left(-a_{31} x_1^{(n+1)} - a_{32} x_2^{(n+1)} - \sum_{j=4}^{n} a_{3j} x_j^{(n)} + b_3 \right) \\ \vdots \end{cases}$$

La matrice G définie par

$$G = K^{-1} M = (D - E)^{-1} F$$

est appelée **matrice de Gauss-Seidel.**

Remarque 6.5.1 *A priori la méthode de Gauss-Seidel devrait être plus performante que la méthode de Jacobi puisqu'on tient compte au fur et à mesure, des valeurs $x_i^{(n+1)}$ déjà calculées.*

6.5.1 Estimation de l'erreur de la méthode de Gauss – Seidel.

Si on écrit $G = (g_{ij})_{i,j=1}^{N}$, alors l'estimation de l'erreur dû à la méthode de Gauss – Seidel vérifie

$$\left\| x - x^{(n)} \right\| \leq \frac{\mu^n}{1 - \mu} \left\| x^{(1)} - x^{(0)} \right\| \tag{5.13}$$

où

$$\mu = \max_{0 \leq i \leq N} \frac{\sum_{j=i}^{N} |g_{ij}|}{1 - \sum_{j=1}^{i-1} |g_{ij}|}.$$

Exemple 6.5.2 *On considère le système*

$$\begin{cases} 5x_1 - x_2 - x_3 - x_4 = -4 \\ -x_1 + 10x_2 - x_3 - x_4 = 12 \\ x_1 - x_2 + 5x_3 - x_4 = 8 \\ x_1 - x_2 - x_3 + 10x_4 = 34 \end{cases} \Leftrightarrow \begin{pmatrix} 5 & -1 & -1 & -1 \\ -1 & 10 & -1 & -1 \\ -1 & -1 & 5 & -1 \\ -1 & -1 & -1 & 10 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} -4 \\ 12 \\ 8 \\ 34 \end{pmatrix}$$

dont la solution est $x = (1, 2, 3, 4)^t$.

(a) Méthode de Jacobi :

La matrice de Jacobi est $J = K^{-1}M = D^{-1}(E + F)$, *où*

$$D = \begin{pmatrix} 5 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 10 \end{pmatrix}, \; E = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{pmatrix} \; et \; F = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

et l'algorithme de Jacobi est $x^{(p+1)} = J.x^{(p)} + D^{-1}b$.

$$\begin{aligned} D^{-1}(E+F) &= \begin{pmatrix} 1/5 & 0 & 0 & 0 \\ 0 & 1/10 & 0 & 0 \\ 0 & 0 & 1/5 & 0 \\ 0 & 0 & 0 & 1/10 \end{pmatrix} \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 0 & 1/5 & 1/5 & 1/5 \\ 1/10 & 0 & 1/10 & 1/10 \\ 1/5 & 15 & 0 & 1/5 \\ 1/10 & 1/10 & 1/10 & 0 \end{pmatrix}. \end{aligned}$$

Donc l'algorithme devient

$$\begin{pmatrix} x_1^{(p+1)} \\ x_2^{(p+1)} \\ x_3^{(p+1)} \\ x_4^{(p+1)} \end{pmatrix} = \begin{pmatrix} 0 & 1/5 & 1/5 & 1/5 \\ 1/10 & 0 & 1/10 & 1/10 \\ 1/5 & 15 & 0 & 1/5 \\ 1/10 & 1/10 & 1/10 & 0 \end{pmatrix} \begin{pmatrix} x_1^{(p)} \\ x_2^{(p)} \\ x_3^{(p)} \\ x_4^{(p)} \end{pmatrix} + \begin{pmatrix} -4/5 \\ 12/10 \\ 8/5 \\ 34/10 \end{pmatrix},$$

ou encore

$$\begin{cases} x_1^{(p+1)} = \dfrac{1}{5}\left(x_2^{(p)} + x_3^{(p)} + x_4^{(p)} - 4\right) \\ x_2^{(p+1)} = \dfrac{1}{10}\left(x_1^{(p)} + x_3^{(p)} + x_4^{(p)} + 12\right) \\ x_3^{(p+1)} = \dfrac{1}{5}\left(x_1^{(p)} + x_2^{(p)} + x_4^{(p)} + 8\right) \\ x_4^{(p+1)} = \dfrac{1}{10}\left(x_1^{(p)} + x_2^{(p)} + x_3^{(p)} + 34\right). \end{cases}$$

(★) ***La convergence de cette méthode.***

La méthode itérative $x^{(n+1)} = \boldsymbol{B}x^{(n)} + C$ *converge si pour une norme matricielle donnée on a* $\|\boldsymbol{B}\| < 1$.

$$\|J\| = \max_n \sum_{m=1}^{4} |j_{nm}| = \max\{3/5, 3/10\} = 3/5 < 1$$

donc la méthode de Jacobi converge.

On peut aussi utiliser le ***Théorème* 6.3.8.**

(★) ***Les itérations***

En partant de l'approximation initiale $x^{(0)} = (0,0,0,0)^t$, *on obtient les itérations suivantes :*

p	0	1	2	3	4	5
$x_1^{(p)}$	0	-0.8	0.44	0.716	0.8828	0.947448
$x_2^{(p)}$	0	1.2	1.62	1.84	1.929	1.96912
$x_3^{(p)}$	0	1.6	2.36	2.732	2.8796	2.948412
$x_4^{(p)}$	0	3.4	3.6	3.842	3.9288	3.96914.

(b) Méthode de Gauss – Seidel :

La matrice de Gauss – Seidel est $G = (D-E)^{-1}F$.

$$D - E = \begin{pmatrix} 5 & 0 & 0 & 0 \\ -1 & 10 & 0 & 0 \\ -1 & -1 & 5 & 0 \\ -1 & -1 & -1 & 10 \end{pmatrix},$$

alors en utilisant la méthode de Jordan on obtient

$$(D-E)^{-1} = \begin{pmatrix} 1/5 & 0 & 0 & 0 \\ 1/50 & 1/10 & 0 & 0 \\ 11/250 & 1/50 & 1/5 & 0 \\ 33/1250 & 3/250 & 1/50 & 1/10 \end{pmatrix}.$$

Donc

$$G = (D-E)^{-1}F = \begin{pmatrix} 0 & 1/5 & 1/5 & 1/5 \\ 0 & 1/50 & 3/25 & 3/25 \\ 0 & 11/250 & 8/125 & 3/125 \\ 0 & 33/1250 & 24/625 & 75/1250 \end{pmatrix}$$

(★) ***La convergence de cette méthode.***

$\|G\| = 3/5 < 1 \Rightarrow$ *la convergence de la méthode de Gauss – Seidel.*

(★) ***L'algorithme de Gauss – Seidel*** *.*

L'algorithme de cette méthode est

$$(D - E)\, x^{(p+1)} = F x^{(p)} + b,$$

ou encore

$$\begin{pmatrix} 5 & 0 & 0 & 0 \\ -1 & 10 & 0 & 0 \\ -1 & -1 & 5 & 0 \\ -1 & -1 & -1 & 10 \end{pmatrix} \begin{pmatrix} x_1^{(p+1)} \\ x_2^{(p+1)} \\ x_3^{(p+1)} \\ x_4^{(p+1)} \end{pmatrix} = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} x_1^{(p)} \\ x_2^{(p)} \\ x_3^{(p)} \\ x_4^{(p)} \end{pmatrix} + \begin{pmatrix} -4 \\ 12 \\ 8 \\ 34 \end{pmatrix},$$

ou encore

$$\begin{cases} 5x_1^{(p+1)} = x_2^{(p)} + x_3^{(p)} + x_4^{(p)} - 4 \\ -x_1^{(p+1)} + 10x_2^{(p+1)} = x_3^{(p)} + x_4^{(p)} + 12 \\ -x_1^{(p+1)} - x_2^{(p+1)} + 5x_3^{(p+1)} = x_4^{(p)} + 8 \\ x_1^{(p+1)} + x_2^{(p+1)} + x_3^{(p+1)} + 10x_4^{(p+1)} = 34. \end{cases}$$

Par conséquent on obtient

$$\begin{cases} x_1^{(p+1)} = \dfrac{1}{5}\left(x_2^{(p)} + x_3^{(p)} + x_4^{(p)} - 4\right) \\ x_2^{(p+1)} = \dfrac{1}{10}\left(x_1^{(p+1)} + x_3^{(p)} + x_4^{(p)} + 12\right) \\ x_3^{(p+1)} = \dfrac{1}{5}\left(x_1^{(p+1)} + x_2^{(p+1)} + x_4^{(p)} + 8\right) \\ x_4^{(p+1)} = \dfrac{1}{10}\left(x_1^{(p+1)} + x_2^{(p+1)} + x_3^{(p+1)} + 34\right). \end{cases}$$

Alors en partant de l'approximation initiale $x^{(0)} = (0, 0, 0, 0)^t$ *on on btient les résultats suivant*

p	0	1	2	3	4
$x_1^{(p)}$	0	-0.8	0.47648	0.889130752	0.97700051
$x_2^{(p)}$	0	1.12	1.7773888	1.956089651	1.990591378
$x_3^{(p)}$	0	1.664	2.7697536	2.949446513	2.989411716
$x_4^{(p)}$	0	3.3.5984	3.90201216	3.979466692	3.99570036.

En comparant les results des deux méthodes avec la valeur exacte de la solution, on peut dire que la méthode de Gauss – Seidel est plus précise que la méthode de Jacobi.

Chapitre 7

Résolution numérique des équations différentielles ordinaires du premier ordre.

7.1 Introduction.

Dans ce chapitre on s'intéresse aux équations différentielles ordinaires du premier ordre de la forme

$$(\text{Pr}) \begin{cases} \text{Trouver la fonction } y \mapsto y(x) \text{ telle que} \\ \dfrac{dy}{dx} = f(x,y); \quad \text{pour tout } x \in [a,b] \\ y(a) = y_0 \text{ donné.} \end{cases} \tag{P}$$

Les problémes différetielles de ce type sont appelés **problèmes de Cauchy** ou **problémes à valeur initiale**.

Si f est une fonction continue vérifiant une condition de Lipschitz (fonction lipschitzienne) par rapport à la deuxième variable y, i. e.

$$\exists M > 0 \text{ telle que } \forall x \in [a,b], \forall y_1, y_2 \text{ on ait } \left| f(x, y_1(x)) - f\left(x, y_2\left(x\right)\right)\right| \leq \left|y_1\left(x\right) - y_2\left(x\right)\right|,$$

alors le problème (Pr) admet une solution unique y pour toute valeur initiale y_0. On dit, dans ce cas, que ce problème est **bien posé**.

En pratique, il s'est avéré qu'il était impossible de trouver les solutions exactes de certains problèmes bien posés du point de vue théorique. Pour cela,

il devient plus commode de chercher des solutions approchées, en utilisant les méthodes numériques conçues pour ce type de problèmes.

Comme les ordinateurs ne peuvent fournir qu'un nombre fini de résultats, c – à – d, qu'on ne peut pas avoir des valeurs approchées de la solution y du problème (P) sur tout l'intervalle $[a, b]$. Tous commence donc par un choix préalable d'un nombre fini de points $x_i \in [a, b]$. Ceci s'appelle une **discrétisation** ou un **maillage** du domaine (ici c'est le segment $[a, b]$). On limitera donc le calcul au calcul approché de la solution en ces points.

On va cité dans ce chapitre quelques méthodes (schémas) d'intégration à un pas, c–à–d, les méthodes qui nous permettent de calculer la valeur approchée de la solution au point x_{i+1}, notée $y_{i+1} = y\left(x_{i+1}\right)$, en fonction de la valeur appopchée de la solution au point x_i notée $y_i = y\left(x_i\right)$.

7.2 Méthode d'Euler.

Discrétisant l'intervalle $[a, b]$ à l'aide des points équidistants $a = x_0 < x_1 < \cdots < x_n = b$, où $x_{i+1} - x_i = h$ pour tout $i = 0, 1, \cdots, n-1$ et $h = \dfrac{b-a}{n}$.

En intégrant par rapport à x les deux cotés de l'equation $\dfrac{dy}{dx} = f(x, y)$ sur l'intervalle $[x_i, x_{i+1}]$, on obtient

$$y\left(x_{i+1}\right) - y\left(x_i\right) = y_{i+1} - yi = \int_{x_i}^{x_{i+1}} dy = \int_{x_i}^{x_{i+1}} f\left(x, y(x)\right) dx,$$

alors, en utilisant la formule de quadrature des réctangles (ou on interpole f par rapport à la variable x par un polynôme constant sur chaque sous intervalle $[x_i, x_{i+1}]$) suivante

$$\int_a^b g(x)dx \approx (b-a)g(\alpha)$$

où $\alpha = a$ ou b ou $\dfrac{a+b}{2}$, on obtient en remplaçant α par a, la **formule d'Euler explicite** suivante

$$y_{i+1} = y_i + hf\left(x_i, y_i\right) \tag{6.1}$$

et si on remplace α par b, on obtient la la **formule d'Euler implicite** suivante

$$y_{i+1} = y_i + hf\left(x_{i+1}, y_{i+1}\right). \tag{6.2}$$

Géomètriquement, la méthode d'Euler explicite revient à remplacer localement en chaque point x_i, la courbe solution par sa tangente. ainsi on va réunir sur le plan des cordonnées les points $(x_0, y_0), (x_1, y_1), \cdots, (x_n, y_n)$ par des segment de droites pour obtenir une ligne brisée qui est la représentation approchée de la courbe d'intégrale. Cette ligne brisée est appelée **ligne brisée d'Euler**.

Remarque 7.2.1 *On peut démontrer que si* (Pr) *admet une solution unique* y *pour toute valeur initiale* y_0*, la méthode d'Euler explicite converge, c–à–d que si on note la solution approchée de ce problème correspondant au pas* h *par* y_h *on obtient*

$$\lim_{h\to 0}\left[y_h(x) - y(x)\right] = 0 \quad \textit{pour tout } x \in [a, b]$$

et que l'erreur $R = y_h(x) - y(x)$ *est d'ordre* 1 *en* h*, i. e.*

$$y_h(x) - y(x) = o(h) \Longleftrightarrow \lim_{h\to 0}\left[\frac{y_h(x) - y(x)}{h}\right] = 0.$$

Exemple 7.2.2 *Trouver la valeur approchée pour* $x = 1$ *de la solution* y *de l'équation*

$$y' = y + x$$

vérifiant la condition initiale $y_0 = 1$ *pour* $x_0 = 0$ *(en prendra* $h = 0.1$ *).*

Solution 7.2.3 *Divisons le segment* $[0, 1]$ *en* 10 *parties à l'aide des points* $x_i = i/10$*; pour* $i = 0, 1, \ldots, 10$ *et cherchons les valeurs de* y_i*, pour* $i = 0, 1, \ldots, n = 10$ *en utilisant la formule d'Euler explicite* (6.1) *qui nous permet d'écrire*

$$\begin{aligned} y_1 &= y_0 + 0.1\,(x_0 + y_0) = 1.1 \\ y_2 &= y_1 + 0.1\,(y_1 + x_1) = 1.2 \\ &\ldots\ldots\ldots\ldots \end{aligned}$$

Au cours de la résolution nous formons le tableau suivant

x_k	y_k	$x_k + y_k$	$(x_k + y_k)\, h$
$x_0 = 0$	1.000	1.000	0.100
$x_1 = 0.1$	1.100	1.200	0.120
$x_2 = 0.2$	1.220	1.420	0.112
$x_{03} = 0.3$	1.362	1.620	0.162
$x_4 = 0.4$	1.524	1.924	0.1924
$x_5 = 0.5$	1.7164	2.2164	0.22164
$x_6 = 0.6$	1.93804	2.53804	0.253804
$x_7 = 0.7$	2.191844	2.891844	0.2891844
$x_8 = 0.8$	2.4810284	3.2810284	0.32810284
$x_9 = 0.9$	2.80913124	3.7091124	0.370913124
$x_{10} = 1.0$	3.180044364		

Nous avons trouver une valeur approchée $dey|_{x=1} = y(1) = 3.1800$.

La solution exacte de cette équation, vérifiant la condition initiale citée précédemment est

$$y = 2\exp x - x - 1 \Rightarrow y(1) = 2\,(e-1) = 3.4366.$$

L'erreur est égale à 0.2566.

★ ***La convergence***

Si f *est une fonction lipschitzienne par rapport à la deuxième variable* y, *la méthode d'euler converge.*

$$|f(x, y_1) - f(x, y_2)| = |y_1 - y_2| \leq |y_1 - y_2|\,.$$

Par conséquent f *est une fonction lipschitzienne par rapport à la deuxième variable* y, *ce qui donne la convergence de la méthode.*

7.3 Méthodes de Runge – Kutta.

Les méthodes de Runge – Kutta sont les généralisations de la méthode d'Euler à des ordres supérieurs à un. Elles s'obtiennent à partir des formules d'intégrations numériques plus précises que la formule des rectangles.

En effet, l'utilisation de la formule des trapèzes

$$\int_{x_i}^{x_{i+1}} g(x)dx \approx \frac{h}{2}\left[g(x_i) + g(x_{i+1})\right]$$

nous conduit à la méthode (schéma) suivante

$$\begin{cases} y_0 \text{ , valeur initiale donnée} \\ y_{i+1} = y_i + \dfrac{h}{2}\left[f\left(x_i, y_i\right) + f\left(x_{i+1}, y_{i+1}\right)\right]. \end{cases} \tag{6.3}$$

Cette méthode est une méthode implicite. Si on veut obtenir une méthode explicite du même ordre, on peut procéder de la manière suivante

$$\begin{cases} y_0 \text{ , valeur initiale donnée} \\ y^*_{i+1} = y_i + hf(x_i, y_i) \\ y_{i+1} = y_i + \dfrac{h}{2}\left[f\left(x_i, y_i\right) + f\left(x_{i+1}, y^*_{i+1}\right)\right]. \end{cases} \tag{6.4}$$

On obtient ainssi la **formule de Runge – Kutta d'odre 2**.

De même, l'utilisation de la formule d'intégration numérique de Simpson est à la base de la formule de **formule de Runge – Kutta d'odre 4**. C'est l'une des formules les plus utilisées, elle s'écrit

$$\begin{cases} y_0 \text{ , valeur initiale donnée} \\ k_1 = hf(x_i, y_i) \\ k_2 = hf\left(x_i + \dfrac{h}{2}, y_i + \dfrac{k_1}{2}\right) \\ k_3 = hf\left(x_i + \dfrac{h}{2}, y_i + \dfrac{k_2}{2}\right) \\ k_4 = hf\left(x_i + \dfrac{h}{2}, y_i + \dfrac{k_3}{2}\right) \\ y_{i+1} = y_i + \dfrac{1}{6}\left[k_1 + 2k_2 + 2k_3 + k_4\right]. \end{cases} \tag{6.5}$$

Exemple 7.3.1 *Trouver la solution y de l'équation*

$$y' = y + x$$

vérifiant la condition initiale $y_0 = 1$ pour $x_0 = 0$, sur l'intervalle $[0, 0.4]$.

Solution 7.3.2 *Prenons* $h = 0.1$.

♣ ***Formule de Runge – Kutta d'odre 2.***

D'après l'algorithme (6.4) *on a*

$$\begin{aligned} y_1^* &= y_0 + hf(x_0, y_0) = y_0 + 0.1(x_0 + y_0) = 1.1 \\ y_1 &= y_0 + \frac{h}{2}\left[f(x_0, y_0) + f(x_1, y_1^*)\right] \\ &= y_0 + 0.05\left[(x_0 + y_0) + (x_1 + y_1^*)\right] = 1.21. \end{aligned}$$

De la même manière, on peut obtenir y_2, y_3 *et* y_4 *donnés dans le tableau suivant*

i	x_i	y_i	y_i^*	$x_i + y_i^*$	$x_i + y_i$
0	0	1			1
1	0.1	1.11	1.1	1.2	1.21
2	0.2	1.24205	1.231	1.431	1.44204
3	0.3	1.39846525	1.386255	1.68255	1.689846525
4	0.4	1.581804101	1.5683117755	1.9683117755	

Donc $y(0.4) \approx 1.5818041 = y_{RK2}$ *par la méthode de Runge–Kutta d'ordre 2*

♣ ***Formule de Runge – Kutta d'odre 4.***

D'après l'algorithme (6.5),*pour calculer* y_1, *on a*

$$\begin{aligned} k_1^{(0)} &= hf(x_0, y_0) = h\left(x_0 + y_0\right) = 0.1 \\ k_2^{(0)} &= hf\left(x_0 + \frac{h}{2}, y_0 + \frac{k_1^{(0)}}{2}\right) = h\left[\left(x_0 + \frac{h}{2}\right) + \left(y_0 + \frac{k_1^{(0)}}{2}\right)\right] \\ &= 0.1\left[(0 + 0.05) + (1 + 0.05)\right] = 0.11, \\ k_3^{(0)} &= hf\left(x_0 + \frac{h}{2}, y_0 + \frac{k_2^{(0)}}{2}\right) = h\left[\left(x_0 + \frac{h}{2}\right) + \left(y_0 + \frac{k_2^{(0)}}{2}\right)\right] \\ &= 0.1\left[(0 + 0.05) + (1 + 0.055)\right] = 0.1105 \quad et \\ k_4^{(0)} &= hf\left(x_0 + h, y_0 + k_3^{(0)}\right) = h\left[(x_0 + h) + \left(y_0 + k_3^{(0)}\right)\right] \\ &= 0.1\left[(0 + 0.1) + (1 + 0.1105)\right] = 0.12105. \end{aligned}$$

Donc

$$y_1 = y_0 + \frac{1}{6}\left[k_1^{(0)} + 2k_2^{(0)} + 2k_3^{(0)} + k_4^{(0)}\right] = 1.11034167.$$

De même pour calculer y_2 on a

$$\begin{aligned}
k_1^{(1)} &= hf(x_1, y_1) = h\left(x_1 + y_1\right) = 0.121034167 \\
k_2^{(1)} &= hf\left(x_1 + \frac{h}{2}, y_1 + \frac{k_1^{(1)}}{2}\right) = h\left[\left(x_1 + \frac{h}{2}\right) + \left(y_1 + \frac{k_1^{(1)}}{2}\right)\right] \\
&= 0.1\left[(0.1 + 0.05) + (1.11034167 + 0.06051708)\right] = 0.13208587 \\
k_3^{(1)} &= hf\left(x_1 + \frac{h}{2}, y_1 + \frac{k_2^{(1)}}{2}\right) = h\left[\left(x_1 + \frac{h}{2}\right) + \left(y_1 + \frac{k_2^{(1)}}{2}\right)\right] \\
&= 0.1\left[(0.1 + 0.05) + (1.11034167 + 0.06604294)\right] = 0.13263846 \\
k_4^{(1)} &= hf\left(x_1 + h, y_1 + k_3^{(1)}\right) = h\left[(x_1 + h) + \left(y_1 + k_3^{(1)}\right)\right] \\
&= 0.1\left[(0.1 + 0.1) + (1.11034167 + 0.13263846)\right] = 0.14429801,
\end{aligned}$$

par conséquent

$$y_2 = y_1 + \frac{1}{6}\left[k_1^{(1)} + 2k_2^{(1)} + 2k_3^{(1)} + k_4^{(1)}\right] = 1.24280514.$$

Pour y_3 on a

$$\begin{aligned}
k_1^{(2)} &= hf(x_2, y_2) = h\left(x_2 + y_2\right) = 0.14428051 \\
k_2^{(2)} &= hf\left(x_2 + \frac{h}{2}, y_2 + \frac{k_1^{(2)}}{2}\right) = h\left[\left(x_2 + \frac{h}{2}\right) + \left(y_2 + \frac{k_1^{(2)}}{2}\right)\right] \\
&= 0.1\left[(0.2 + 0.05) + (1.242804514 + 0.07214026)\right] = 0.15649448 \\
k_3^{(2)} &= hf\left(x_2 + \frac{h}{2}, y_2 + \frac{k_2^{(2)}}{2}\right) = h\left[\left(x_2 + \frac{h}{2}\right) + \left(y_2 + \frac{k_2^{(2)}}{2}\right)\right] \\
&= 0.1\left[(0.2 + 0.05) + (1.24280514 + 0.07824724)\right] = 0.15710524 \\
k_4^{(2)} &= hf\left(x_2 + h, y_2 + k_3^{(2)}\right) = h\left[(x_2 + h) + \left(y_2 + k_3^{(2)}\right)\right] \\
&= 0.1\left[(0.2 + 0.1) + (1.24280514 + 0.15710524)\right] = 0.16999104,
\end{aligned}$$

par conséquent

$$y_3 = y_2 + \frac{1}{6}\left[k_1^{(2)} + 2k_2^{(2)} + 2k_3^{(2)} + k_4^{(2)}\right] = 1.39971697$$

En fin Pour y_4 on a

$$\begin{aligned}
k_1^{(3)} &= hf(x_3, y_3) = h\,(x_3 + y_3) = 0.1699717 \\
k_2^{(3)} &= hf\left(x_3 + \frac{h}{2}, y_3 + \frac{k_1^{(3)}}{2}\right) = h\left[\left(x_3 + \frac{h}{2}\right) + \left(y_3 + \frac{k_1^{(3)}}{2}\right)\right] \\
&= 0.1\left[(0.3 + 0.05) + 1.39971697 + 0.08498585\right] = 0.18347028 \\
k_3^{(3)} &= hf\left(x_3 + \frac{h}{2}, y_3 + \frac{k_2^{(3)}}{2}\right) = h\left[\left(x_3 + \frac{h}{2}\right) + \left(y_3 + \frac{k_2^{(3)}}{2}\right)\right] \\
&= 0.1\left[(0.3 + 0.05) + (1.39971697 + 0.09173514)\right] = 0.18414521 \\
k_4^{(3)} &= hf\left(x_3 + h, y_3 + k_3^{(3)}\right) = h\left[(x_3 + h) + \left(y_3 + k_3^{(3)}\right)\right] \\
&= 0.1\left[(0.3 + 0.1) + (1.39971697 + 0.118414521)\right] = 0.19838622
\end{aligned}$$

par conséquent

$$y_4 = y_3 + \frac{1}{6}\left[k_1^{(3)} + 2k_2^{(3)} + 2k_3^{(3)} + k_4^{(3)}\right] = 1.58364845.$$

Donc $y(0.4) \approx 1.5836485 = y_{RK4}$ *par la méthode de Runge–Kutta d'ordre* 4

La solution exacte de cette équation vérifiant la condition initiale citée précédemment est

$$y = 2\exp x - x - 1 \Rightarrow y(0.4) = 1.583649395.$$

On remarque que

$$\begin{aligned}
|y(0.4) - y_{RK2}| &\approx 0.2 \times 10^{-2} \Rightarrow \textit{valeur approchée avec au total 3 c.s.e.} \\
|y(0.4) - y_{RK4}| &\approx 0.9 \times 10^{-6} \Rightarrow \textit{valeur approchée avec au total 6 c.s.e.}
\end{aligned}$$